111Equation Chapter 1 Section 1TỔNG LIÊN ĐOÀN LAO ĐỘNG VIỆT NAM

TRƯỜNG ĐẠI HỌC TÔN ĐỨC THẮNG

KHOA ĐIỆN – ĐIỆN TỬ

ĐỖ MINH CHÍNH

ĐIỀU KHIỂN TỐC ĐỘ ĐỘNG CƠ SỬ DỤNG PHƯƠNG PHÁP PID DÙNG STM32

ĐỒ ÁN CHUYÊN NGÀNH
KỸ THUẬT ĐIỀU KHIỂN VÀ TỰ ĐÔNG HÓA

Người hướng dẫn: **TS. NGUYỄN THỊ PHƯƠNG THẢO**

Họ và tên: **ĐỖ MINH CHÍNH**

Mã số sinh viên: 42001128

Lớp: 20040301

Khóa: 24

THÀNH PHỐ HỒ CHÍ MINH, NĂM 2023

TỔNG LIÊN ĐOÀN LAO ĐỘNG VIỆT NAM
TRƯỜNG ĐẠI HỌC TÔN ĐỨC THẮNG
KHOA ĐIỆN – ĐIỆN TỬ



ĐỖ MINH CHÍNH

ĐIỀU KHIỂN TỐC ĐỘ ĐỘNG CƠ SỬ DỤNG PHƯƠNG PHÁP PID DÙNG STM32

ĐỒ ÁN CHUYÊN NGÀNH
KỸ THUẬT ĐIỀU KHIỂN VÀ TỰ ĐÔNG HÓA

Người hướng dẫn: **TS. NGUYỄN THỊ PHƯƠNG THẢO**

Họ và tên: **ĐỖ MINH CHÍNH**

Mã số sinh viên: 42001128

Lớp: 20040301

Khóa: 24

THÀNH PHỐ HỒ CHÍ MINH, NĂM 2023

LỜI CẢM ƠN

Chúng em xin gởi lời cảm ơn chân thành và lòng biết ơn sâu sắc nhất đối với Cô Nguyễn Thị Phương Thảo, Cô đã nhiệt tình truyền đạt các kiến thức, kinh nghiệm bổ ích và giúp đỡ em nhiệt tình trong thời gian thực hiện báo cáo. Những lời nhận xét và góp ý của Cô đã tạo thành các tiền đề quan trọng giúp chúng em có nhận thức đúng, chỉ rõ những ưu khuyết điểm của bản thân và biết nhìn nhận và xử lý vấn đề một cách tốt nhất. Nhờ vậy mà chúng em đã hoàn thành bài thuyết trình của mình một cách hoàn hảo hơn.

Mặc dù đã cố gắng hết sức để hoàn thiện báo cáo đồ án hệ thống nhúng theo phạm vi và khả năng bản thân. Tuy nhiên, chúng em sẽ không tránh được một số thiếu sót và sai lầm. Em rất mong có được sự đồng cảm và chỉ bảo nhiệt tình của Cô.

Xin trân trọng cảm ơn Cô!

TP. Hồ Chí Minh, ngày 29 tháng 11 năm 2023

Tác giả

Đỗ Minh Chính

Công trình được hoàn thành tại Trường Đại học Tôn Đức Thắng

Cán bộ hướng dẫn khoa học:

.....

.....

(Ghi rõ học hàm, học vị, họ tên và chữ ký)

Đồ án tốt nghiệp/tổng hợp được bảo vệ tại **Hội đồng đánh giá Đồ án tốt nghiệp/tổng hợp của Trường Đại học Tôn Đức Thắng** vào ngày… /…/……

Xác nhận của Chủ tịch Hội đồng đánh giá Đồ án tốt nghiệp/tổng hợp và Trưởng khoa quản lý chuyên ngành sau khi nhận Đồ án tốt nghiệp/tổng hợp đã được sửa chữa (nếu có).

CHỦ TỊCH HỘI ĐỒNG	**TRƯỞNG KHOA**
……	**……**

BÁO CÁO ĐỒ ÁN CHUYÊN NGÀNH ĐƯỢC HOÀN THÀNH TẠI TRƯỜNG ĐẠI HỌC TÔN ĐỨC THẮNG

Em xin cam đoan đây là công trình nghiên cứu và được sự hướng dẫn từ Cô Võ Hữu Hậu. Các nội dung nghiên cứu, kết quả trong đề tài này là trung thực và chưa công bố dưới bất kỳ hình thức nào trước đây. Những số liệu trong các bảng biểu phục vụ cho việc phân tích, nhận xét, đánh giá được chính tác giả thu thập từ các nguồn khác nhau có ghi rõ trong phần tài liệu tham khảo.

Ngoài ra, trong báo cáo còn sử dụng một số nhận xét, đánh giá cũng như số liệu của các tác giả khác, cơ quan tổ chức khác đều có trích dẫn và chú thích nguồn gốc.

Nếu phát hiện có bất kỳ sự gian lận nào em xin hoàn toàn chịu trách nhiệm về nội dung Đồ án hệ thống nhúng của mình. Trường Đại học Tôn Đức Thắng không liên quan đến những vi phạm tác quyền, bản quyền do tôi gây ra trong quá trình thực hiện (nếu có).

TP. Hồ Chí Minh, ngày 29 tháng 11 năm 2023

Tác giả

Đỗ Minh Chính

TÓM TẮT

Tóm tắt đề tài điều khiển tốc độ động cơ bằng giải thuật PID dùng STM32

Động cơ DC là một trong những loại động cơ được sử dụng phổ biến nhất trong các hệ thống tự động hóa. Để điều khiển tốc độ của động cơ DC, có nhiều phương pháp khác nhau, trong đó phương pháp sử dụng giải thuật PID là một trong những phương pháp phổ biến nhất.

Trong đề tài này, tác giả đã đề xuất một mạch điều khiển tốc độ động cơ DC sử dụng giải thuật PID trên nền tảng vi điều khiển STM32. Mạch điều khiển bao gồm các thành phần chính sau:

- Vi điều khiển STM32: Là bộ điều khiển trung tâm của mạch, thực hiện tính toán và điều khiển tốc độ động cơ.
- Động cơ DC: Là đối tượng được điều khiển.
- Cảm biến encoder: Đo tốc độ quay của động cơ.
- Bộ điều chỉnh PID: Thực hiện điều chỉnh tốc độ động cơ theo sai số giữa tốc độ mong muốn và tốc độ thực tế.

Mạch hoạt động theo nguyên lý sau:

- Vi điều khiển nhận tín hiệu tốc độ mong muốn từ người dùng.
- Vi điều khiển sử dụng cảm biến encoder để đo tốc độ thực tế của động cơ.
- Vi điều khiển tính toán sai số giữa tốc độ mong muốn và tốc độ thực tế.
- Vi điều khiển sử dụng bộ điều chỉnh PID để tính toán tín hiệu điều khiển.
- Tín hiệu điều khiển được đưa đến động cơ để điều chỉnh tốc độ quay của động cơ.

Để thực hiện đề tài, tác giả đã thực hiện các bước sau:

- Khảo sát các loại vi điều khiển, lựa chọn động cơ DC, khảo sát các linh kiện liên quan để tiến hành thiết kế mạch.

- Tham khảo các mô hình điều khiển động cơ để xây dựng lên mô hình điều khiển.
- Nghiên cứu tài liệu về bộ điều chỉnh PID DRIVER để điều khiển tốc độ của động cơ DC.
- Thiết kế mạch điều khiển trên phần mềm STM32CubeMX.
- Lập trình vi điều khiển.
- Thực nghiệm và đánh giá kết quả.
- Kết quả thực nghiệm cho thấy mạch điều khiển hoạt động ổn định và đáp ứng được các yêu cầu đặt ra. Tốc độ động cơ có thể điều chỉnh chính xác theo tín hiệu mong muốn.

Đề tài có thể ứng dụng trong các hệ thống tự động hóa cần điều khiển tốc độ động cơ DC, chẳng hạn như hệ thống robot, hệ thống điều khiển thiết bị công nghiệp,...

Kết luận: Đề tài điều khiển tốc độ động cơ DC bằng giải thuật PID dùng STM32 là một đề tài có ý nghĩa thực tiễn. Mạch điều khiển được thiết kế với tính ổn định và độ chính xác cao, có thể ứng dụng trong nhiều hệ thống tự động hóa.

MỤC LỤC

MỤC LỤC VI

DANH MỤC HÌNH ẢNH.....VIII

DANH MỤC BẢNG BIỂU.....IX

DANH MỤC CÁC CHỮ VIẾT TẮT.....X

CHƯƠNG 1. TỔNG QUAN VỀ ĐỀ TÀI 11

1.1 GIỚI THIỆU ĐỀ TÀI.....11

1.2 MỤC ĐÍCH NGHIÊN CỨU.....12

1.3 ĐỐI TƯỢNG NGHIÊN CỨU.....12

1.4 PHẠM VI NGHIÊN CỨU.....12

1.5 DỰ KIẾN KẾT QUẢ.....13

CHƯƠNG 2. MÔ HÌNH TOÁN CỦA ĐỘNG CƠ DC.....14

2.1 GIỚI THIỆU ĐỘNG CƠ DC.....14

2.1.1 Cấu tạo.....14

2.1.2 Nguyên lý hoạt động.....15

2.1.3 Một số loại động cơ DC phổ biến.....16

2.2 THIẾT LẬP VẬT LÝ.....16

2.3 PHƯƠNG TRÌNH HỆ THỐNG.....17

2.4 HÀM TRUYỀN.....18

CHƯƠNG 3. GIẢI THUẬT ĐIỀU KHIỂN PID.....19

3.1 KHÂU TỶ LỆ (PROPORTIONAL).....20

3.2 KHÂU VI PHÂN (INTEGRAL).....21

3.3 KHÂU ĐẠO HÀM (DERIVATIVE).....22

CHƯƠNG 4. PHƯƠNG PHÁP ĐIỀU KHIỂN TỐC ĐỘ ĐỘNG CƠ.....24

4.1 ĐIỀU KHIỂN TỐC ĐỘ ĐỘNG CƠ THEO PHƯƠNG PHÁP ĐIỀU CHẾ ĐỘ RỘNG XUNG PWM 24

4.2 LẬP TRÌNH STM32 VỚI PWM.....25

4.2.1 Giới thiệu STM32F103C8T6.....25
4.2.2 Cấu hình cho vi điều khiển.....28
CHƯƠNG 5. THIẾT KẾ MÔ HÌNH.....32
5.1 SƠ ĐỒ KHỐI CỦA HỆ THỐNG.....32
5.2 LƯU ĐỒ GIẢI THUẬT.....34
5.3 CHƯƠNG TRÌNH LẬP TRÌNH.....35
5.3.1 Thư viện LCD.....35
5.3.2 Thư viện PID.....37
5.3.3 Chương trình chính.....39
TÀI LIỆU THAM KHẢO.....56

DANH MỤC HÌNH ẢNH

Hình 2-1 DC motor construction (courtesy of Baldor Electric Company).....14
Hình 2-2 Cấu tạo đông cơ DC.....14
Hình 3-1 Sơ đồ khối bộ điều khiển PID.....19
Hình 3-2 PID Controller.....20
Hình 3-3 Sơ đồ khối khâu tỷ lệ.....20
Hình 3-4 biểu đồ biểu diễn giá trị ngõ ra hiện tại với các hê số Kp khác nhau.....21
Hình 3-5 Sơ đồ khối khâu tích phân.....21
Hình 3-6 Sơ đồ khối khâu tích phân.....22
Hình 4-1 Sơ đồ module STM32F103C6T8.....25
Hình 4-2 Cấu hình chân STM32F103C6T8.....28
Hình 6-1 Sơ đồ khối của hệ thống.....32
Hình 6-2 Sơ đồ nguyên lý module L298.....33
Hình 6-3 Lưu đồ giải thuật.....34

DANH MỤC CÁC CHỮ VIẾT TẮT

PID	Proportional Integral Derivative Controller (vi tích phân tỷ lệ)
PWM	Pulse-width modulation (Điều chế độ rộng xung)
UART	Universal Asynchronous Receiver-Transmitter (truyền thông nối tiếp bất đồng bộ)
SP	Setpoint
PV	Present value
LCD	Liquid Crystal Display (Màn hình tinh thể lỏng).
IC	Integrated circuit (vi mạch, chip).
I2C	Inter-Intergrated Circuit (Bus nối tiếp).
AC	Alterating Current (dòng điện xoay chiều).
DC	Direct Current (dòng điện một chiều).
GND	Ground (đất).

CHƯƠNG 1. 211Equation Chapter 1 Section 1 TỔNG QUAN VỀ ĐỀ TÀI

1.1 Giới thiệu đề tài

Ngày nay, với sự phát không ngừng của các ngành công nghiệp, nhu cầu sử dụng các máy móc, phương tiện, công cụ sản xuất cũng không ngừng tăng. Để áp ứng nhu cầu tạo ra các tư liệu sản xuất ấy, ta cần một thứ tất yếu, cốt lõi không thể thiếu có thể coi là nền tảng của ngành công nghiệp, đó chính là động cơ. Có 2 loại động cơ chính, đó là động cơ đốt trong và động cơ điện, nhưng vì hiệu suất và thân thiện với môi trường, động cơ điện được sử dụng rộng rãi hơn.

Hiện nay, có rất nhiều loại động cơ điện được sử dụng trong đời sống. Ví dụ: Động cơ đồng bộ, không đồng bộ; động cơ xoay chiều, một chiều; động cơ servo; động cơ giảm tốc; động cơ bước;… Với xu hướng hiện nay, động cơ một chiều dường như được ưu chuộng hơn. Một minh chứng rõ ràng nhất đó là sự chạy đua xe điện ở các nước trên thế giới.

Tình trạng ô nhiễm không khí hiện nay ngày càng tăng cao đặc biệt là ở các khu đô thị, thành phố lớn. Trong đó ô nhiễm giao thông được các cơ quan môi trường đánh giá là một trong những nguyên nhân chính dẫn tới tình trạng này. Cụ thể theo kết quả nghiên cứu của các chuyên gia, hoạt động giao thông sẽ gây ra gần 85% lượng khí cacbon monoxit thải ra không khí mỗi ngày. Với tỷ lệ này, nó có thể gây ảnh hưởng xấu tới cơ thể, thậm chí dẫn tới bị nhiễm độc cấp.

Ngoài ra, việc khai thác và sử dụng các nhiên liệu hóa thạch (xăng, dầu,...) quá mức cũng khiến cho nguồn tài nguyên bị cạn kiệt. Vì vậy, việc tìm kiếm và chuyển đổi sang một nguyên liệu khác thân thiện hơn với môi trường trở nên bức thiết hơn bao

giờ hết. Theo đó, các mẫu xe xanh bao gồm xe ô tô điện, xe bus và xe máy điện đang được xem là giải pháp tối ưu và hiệu quả trong việc bảo vệ môi trường.

Ngoài ra, ứng dụng của động cơ điện một chiều hiện nay cũng rất đa dạng, chúng có mặt trong mọi lĩnh vực sản xuất công nghiệp và đời sống: dùng trong đài FM, ổ đĩa DC, trong tivi, máy công nghiệp, các loại máy in, máy photo,… Đặc biệt là chế tạo Robot.

Những lợi ích mà động cơ điện một chiều mang lại dường như rất to lớn và thiết thực. Nhưng có một bài toán muôn thuở được đặt ra trong việc tận dụng tối đa tài nguyên do động cơ một chiều mang lại, đó là bài toán điều khiển. Điều khiển sao cho chính xác với yêu cầu? Điều khiển sao cho đạt hiệu suất cao nhất? Điều khiển sao cho duy trì tuổi thọ của động cơ?... Đó cũng chính là vấn đề chính của đề tài của em.

1.2 Mục đích nghiên cứu

Em chọn đề tài này nhằm tìm hiểu nguyên lý hoạt hoạt động của động cơ DC, cách dùng vi điều khiển STM32 để tạo một tín hiệu điều khiển phù hợp với tải không cùng mức nguồn cấp của vi điều khiển. Quan trọng nhất đó là điều khiển tốc độ động cơ theo tốc độ cài đặt ban đầu với sai số nhỏ, hệ thống hoạt động ổn định. Ngoài ra, có thể nghiên cứu, tìm hiểu giải thuật điều khiển PID và từ đó vận dụng cho các hệ thống điều khiển tự động khác.

Từ đó, thiết kế các sản phẩm, dự án điều khiển tự động khác trong đời sống.

1.3 Đối tượng nghiên cứu

Vi điều khiển STM32F103C8T6; nguyên lý động cơ DC GA25, encoder; hệ thống điều khiển vòng kín, giải thuật điều khiển PID, giao tiếp truyền thông nối tiếp dữ liệu UART, điều chế độ rộng xung PWM

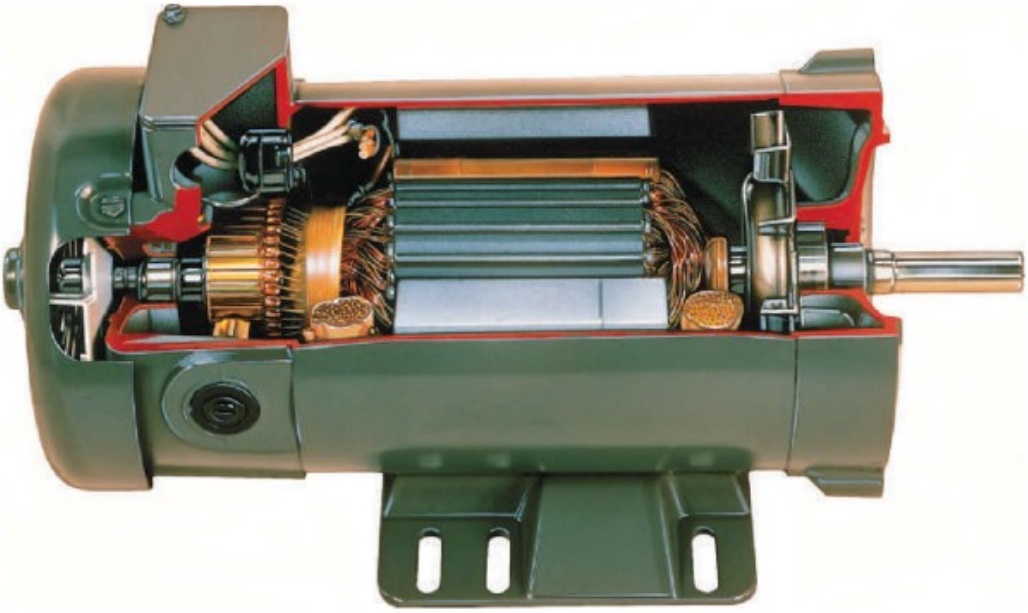
1.4 Phạm vi nghiên cứu

Bộ điều khiển tốc độ động cơ DC: Bộ điều khiển này sử dụng một giải thuật điều khiển PID để điều khiển tốc độ của động cơ DC. Nó có thể được sử dụng để điều khiển động cơ DC có các loại tải khác nhau, bao gồm tải thuần trở, tải cảm ứng và tải trở. Đề tài này có thể được ứng dụng cho các dự án yêu cầu điều khiển tốc độ chính xác, ổn đinh cao, chẳng hạn như băng chuyền, hệ thống tự động hàng hoá, như robot, các dây chuyền sản xuất, các ứng dụng gia dụng như điều khiển quạt, máy bơm,…

1.5 Dự kiến kết quả

Bộ điều khiển tốc độ động cơ là một thiết bị điện tử dùng để điều khiển tốc độ của động cơ. Người dùng sẽ nhập vào tốc độ mong muốn, từ đó bộ điều khiển sẽ sử dụng giải thuật điều khiển để tính toán tín hiệu điều khiển cần thiết để đạt được tốc độ mong muốn. Người dùng gia tiếp qua màn hình LCD, tốc độ đo được từ encoder sẽ phản hồi liên lục và hiển thị lên LCD

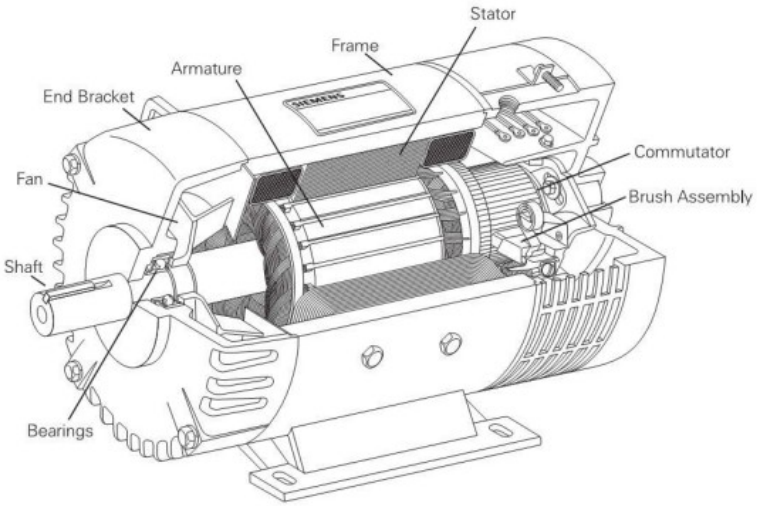
CHƯƠNG 2. MÔ HÌNH TOÁN CỦA ĐỘNG CƠ DC



Hình 2-1 DC motor construction (courtesy of Baldor Electric Company)

2.1 Giới thiệu động cơ DC

2.1.1 Cấu tạo



Hình 2-2 Cấu tạo đông cơ DC

Động cơ DC (viết tắt của Direct Current Motors) là động cơ điện một chiều (dòng điện có hướng xác định) chuyển đổi năng lượng điện thành năng lượng cơ học.
Động cơ DC có cấu tạo gồm hai phần chính:

- Stato: là phần tĩnh, bao gồm các cuộn dây stato.
- Rotor: là phần quay, bao gồm các thanh dẫn hoặc nam châm vĩnh cửu.

2.1.2 Nguyên lý hoạt động

Khi cấp điện cho các cuộn dây stato, sẽ tạo ra một từ trường quay. Từ trường này tác động lên các thanh dẫn hoặc nam châm vĩnh cửu trên rotor, khiến rotor quay.

Cụ thể, khi dòng điện chạy qua các cuộn dây stato, sẽ tạo ra một từ trường. Từ trường này sẽ tác động lên các thanh dẫn trên rotor, tạo ra một lực điện từ. Lực điện từ này sẽ làm cho các thanh dẫn quay.

Hướng quay của rotor phụ thuộc vào hướng dòng điện chạy qua các cuộn dây stato. Nếu dòng điện chạy qua các cuộn dây stato theo chiều kim đồng hồ, thì rotor sẽ quay theo chiều kim đồng hồ. Ngược lại, nếu dòng điện chạy qua các cuộn dây stato theo chiều ngược kim đồng hồ, thì rotor sẽ quay theo chiều ngược kim đồng hồ.

Tốc độ của động cơ DC được xác định bởi điện áp cấp cho động cơ và điện trở của dây quấn stato. Điện áp cấp cho động cơ càng cao thì tốc độ của động cơ càng lớn. Điện trở của dây quấn stato càng cao thì tốc độ của động cơ càng nhỏ.

Động cơ DC có thể được điều chỉnh tốc độ bằng cách thay đổi điện áp cấp cho động cơ. Ví dụ, nếu điện áp cấp cho động cơ giảm một nửa, thì tốc độ của động cơ cũng sẽ giảm một nửa.

Ngoài ra, động cơ DC cũng có thể được điều chỉnh tốc độ bằng cách thay đổi điện trở của dây quấn stato. Ví dụ, nếu điện trở của dây quấn stato tăng gấp đôi, thì tốc độ của động cơ cũng sẽ giảm một nửa.

Động cơ DC có nhiều ưu điểm, bao gồm:

- Khả năng điều chỉnh tốc độ linh hoạt: Động cơ DC có thể được điều chỉnh tốc độ một cách dễ dàng bằng cách thay đổi điện áp cấp cho động cơ.
- Khả năng khởi động và dừng nhanh: Động cơ DC có thể khởi động và dừng nhanh mà không gây ra hiện tượng sốc điện.
- Khả năng chịu tải cao: Động cơ DC có thể chịu tải cao mà không bị quá tải.

Động cơ DC được sử dụng trong nhiều ứng dụng khác nhau, bao gồm:

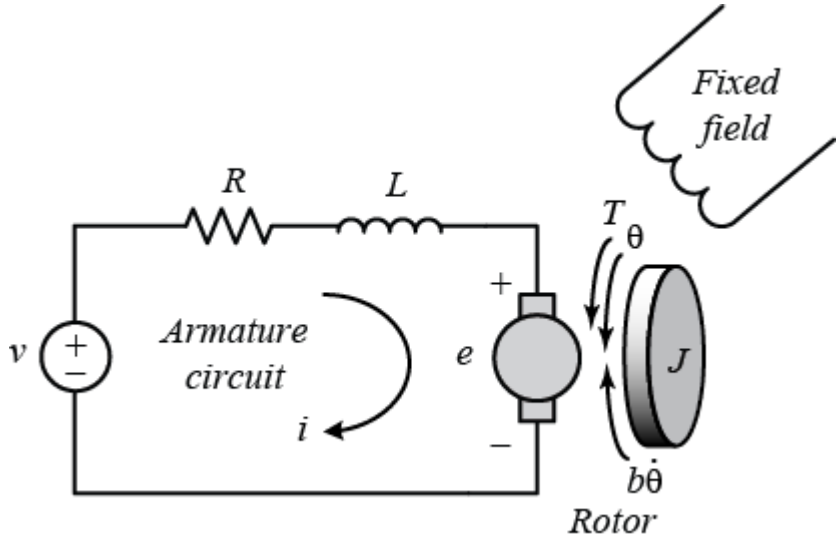
- Các thiết bị gia dụng: Quạt, máy bơm, máy khoan, v.v.
- Các thiết bị công nghiệp: Máy móc, băng tải, robot, v.v.
- Các thiết bị tự động hóa: Hệ thống điều khiển, máy CNC, v.v.

2.1.3 Một số loại động cơ DC phổ biến

- Động cơ DC chổi than: là loại động cơ DC phổ biến nhất. Nó có cấu tạo đơn giản và giá thành rẻ.
- Động cơ DC không chổi than: là loại động cơ DC không sử dụng chổi than. Nó có hiệu suất cao và độ bền cao hơn động cơ DC chổi than.
- Động cơ DC bước: là loại động cơ DC có thể quay từng bước. Nó được sử dụng trong các ứng dụng cần độ chính xác cao, như máy in 3D, máy CNC, v.v.

2.2 Thiết lập vật lý

Một thiết bị truyền động phổ biến trong các hệ thống điều khiển là động cơ DC. Nó trực tiếp cung cấp chuyển động quay và kết hợp với bánh xe hoặc trống và dây cáp, có thể cung cấp chuyển động tịnh tiến. Mạch điện tương đương của phần ứng và sơ đồ phần tự do của rôto được biểu diễn trong hình sau:



Đối với ví dụ này, chúng ta sẽ giả sử rằng đầu vào của hệ thống là nguồn điện áp (V) đặt vào phần ứng của động cơ, trong khi đầu ra là tốc độ quay của trục . Rôto và trục được coi là đứng yên. Chúng tôi tiếp tục giả sử một mô hình ma sát nhớt, nghĩa là mômen ma sát tỷ lệ với vận tốc góc của trục. Các tham số vật lý cho ví dụ của chúng tôi là:

(J)	Momen quán tính của rotor	0.01 kg.m^2
(b)	Hằng số ma sát nhớt động cơ	0.1 N.m.s
(Ke)	Hằng số suất điện động	0.03 V/rad/sec
(Kt)	Hằng số mô-men xoắn của động cơ	0.03 N.m/Amp
(R)	Điện trở	1 Ohm
(L)	Điện cảm	0.5 H

2.3 Phương trình hệ thống

Nói chung, mô-men xoắn do động cơ DC tạo ra tỷ lệ thuận với dòng điện phần ứng và cường độ của từ trường. Trong ví dụ này, chúng ta sẽ giả định rằng từ trường là không đổi và do đó, mô-men xoắn của động cơ chỉ tỷ lệ với dòng điện phần ứng I theo hệ số không đổi Kt như trong phương trình bên dưới. Điều này được gọi là một động cơ điều khiển phần ứng.

$$T = K_t i$$

Emf phía sau, e, tỷ lệ với vận tốc góc của trục theo hệ số không đổi Ke.

$$e = K_e\dot{\theta}$$

Trong các đơn vị SI, các hằng số mô-men xoắn và emf ngược của động cơ bằng nhau, nghĩa là Kt=Ke do đó, chúng tôi sẽ sử dụng K để biểu thị cả hằng số mô-men xoắn của động cơ và hằng số emf phía sau.

Từ hình trên, chúng ta có thể suy ra các phương trình điều chỉnh sau dựa trên định luật 2 Newton và định luật điện áp Kirchhoff.

$$J\ddot{\theta} + b\dot{\theta} = Ki$$

$$L\frac{di}{dt} + Ri = V - K\dot{\theta}$$

2.4 Hàm truyền

Áp dụng phép biến đổi Laplace, các phương trình mô hình hóa trên có thể được biểu diễn dưới dạng biến Laplace s.

$$s(Js + b)\Theta(s) = KI(s)$$

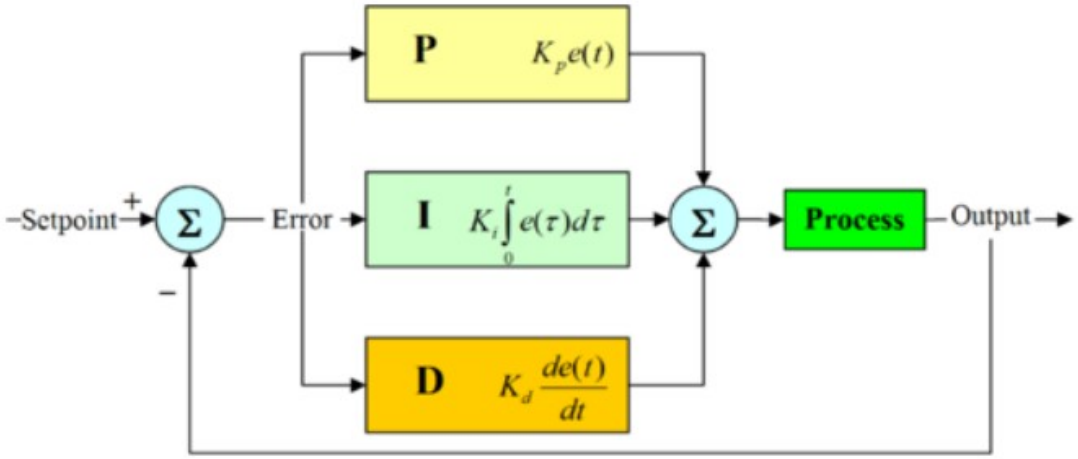
$$(Ls + R)I(s) = V(s) - Ks\Theta(s)$$

Chúng ta đi đến hàm truyền vòng hở sau đây bằng cách loại bỏ I(s) giữa hai phương trình trên, trong đó tốc độ quay được coi là đầu ra và điện áp phần ứng được coi là đầu vào.

$$P(s) = \frac{\dot{\Theta}(s)}{V(s)} = \frac{K}{(Js + b)(Ls + R) + K^2} \qquad [\frac{rad/sec}{V}]$$

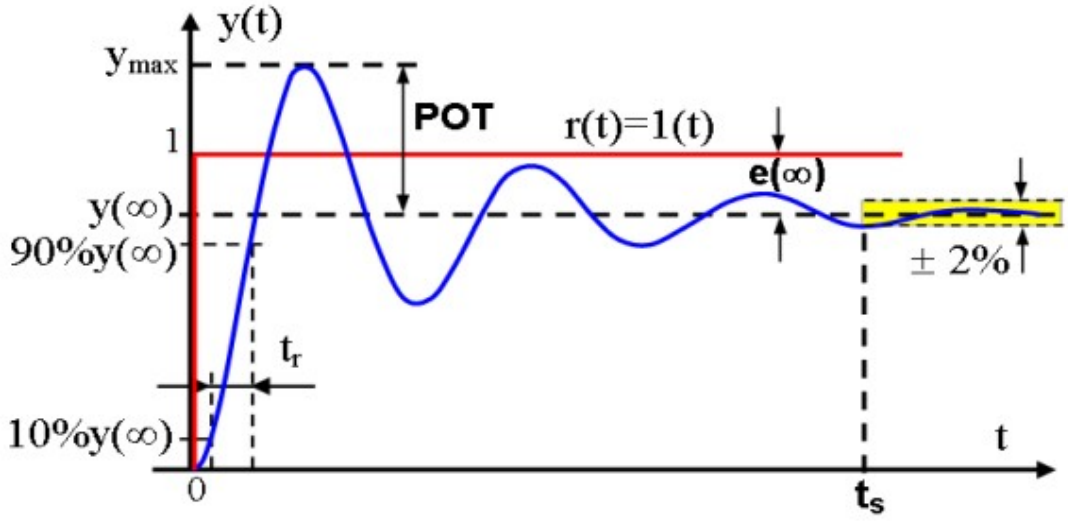
CHƯƠNG 3. GIẢI THUẬT ĐIỀU KHIỂN PID

Có rất nhiều phương pháp điều khiển như: giải thuật điều khiển PID, giải thuật điều khiển tuyến tính, giải thuật điều khiển phi tuyến, giải thuật điều khiển tối ưu, giải thuật điều khiển học máy. Nhưng trong đó, giải thuật được giới điều khiển tự động nghiên cứu, vận nhiều nhất có lẽ là PID.



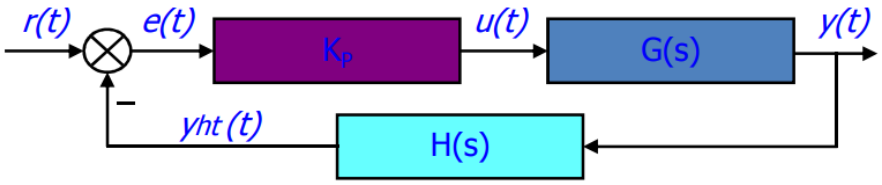
Hình 3-3 Sơ đồ khối bộ điều khiển PID

PID (Proportional Integral Derivative Controller) là bộ điều khiển hồi tiếp vòng kín được sử dụng nhiều nhất trong công nghiệp. Là sự kết hợp của 3 bộ điều khiển : tỉ lệ, tích phân và vi phân. Có khả năng làm triệt tiêu sai số xác lập, tăng tốc độ đáp ứng, giảm độ vọt lố, hạn chế được dao động nếu thông số của bộ điều khiển được lựa chọn thích hợp.



Hình 3-4 PID Controller

3.1 Khâu tỷ lệ (Proportional)



Hình 3-5 Sơ đồ khối khâu tỷ lệ

Khâu tỉ lệ (đôi khi còn được gọi là độ lợi) làm thay đổi giá trị đầu ra, tỉ lệ với giá trị sai số hiện tại. Đáp ứng tỉ lệ có thể được điều chỉnh bằng cách nhân sai số đó với một hằng số Kp, được gọi là hệ số tỉ lệ.

- Hàm truyền:

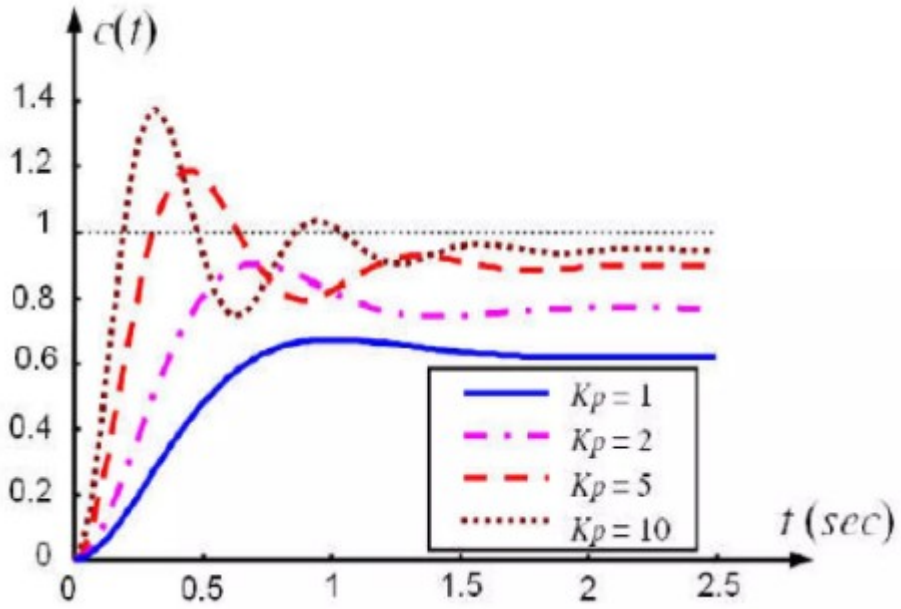
$$K(s)=K_p$$

- Đặc tính thời gian:

$$Y(s)=K_p.G(s).E(s).$$

- Sai số hệ thống:

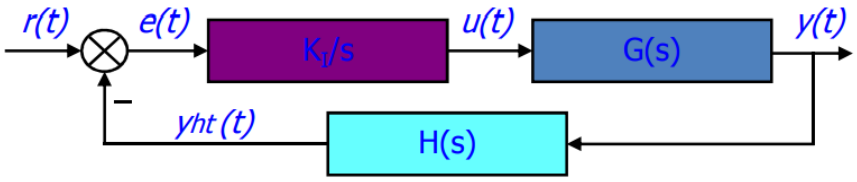
$$E(s)=\frac{R(s)}{1+K_pG(s)H(s)}$$



Hình 3-6 biểu đồ biểu diễn giá trị ngõ ra hiện tại với các hê số Kp khác nhau

Hệ số của khâu tỉ lệ lớn là do thay đổi lớn ở đầu ra mà sai số thay đổi nhỏ. Nếu hệ số của khâu tỉ lệ quá cao, hệ thống sẽ không ổn định (xem phần điều chỉnh vòng). Ngược lại, hệ số nhỏ là do đáp ứng đầu ra nhỏ trong khi sai số đầu vào lớn, và làm cho bộ điều khiển kém nhạy, hoặc đáp ứng chậm. Nếu Hệ số của khâu tỉ lệ quá thấp, tác động điều khiển có thể sẽ quá bé khi đáp ứng với các nhiễu của hệ thống.

3.2 Khâu vi phân (Integral)



Hình 3-7 Sơ đồ khối khâu tích phân

Phân phối của khâu tích phân (đôi khi còn gọi là reset) tỉ lệ thuận với cả biên độ sai số lẫn quảng thời gian xảy ra sai số. Tổng sai số tức thời theo thời gian (tích phân sai số) cho ta tích lũy bù đã được hiệu chỉnh trước đó. Tích lũy sai số sau đó được nhân với độ lợi tích phân và cộng với tín hiệu đầu ra của bộ

điều khiển. Biên độ phân phối của khâu tích phân trên tất cả tác động điều chỉnh được xác định bởi độ lợi tích phân K_i.

- Hàm truyền:

$$K(s)=\frac{K_i}{s}$$

- Đặc tính thời gian:

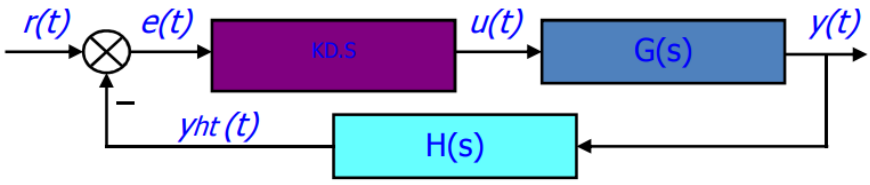
$$Y(s)=K_s.G(s).\frac{E(s)}{s}$$

- Sai số hệ thống:

$$E(s)=\frac{sR(s)}{1+K_iG(s)H(s)}$$

Khâu tích phân (khi cộng thêm khâu tỉ lệ) sẽ tăng tốc chuyển động của quá trình tới điểm đặt và khử số dư sai số ổn định với một tỉ lệ chỉ phụ thuộc vào bộ điều khiển. Tuy nhiên, vì khâu tích phân là đáp ứng của sai số tích lũy trong quá khứ, nó có thể khiến giá trị hiện tại vọt lố qua giá trị đặt (ngang qua điểm đặt và tạo ra một độ lệch với các hướng khác). Để tìm hiểu thêm các đặc điểm của việc điều chỉnh độ lợi tích phân và độ ổn của bộ điều khiển, xin xem phần điều chỉnh vòng lặp.

3.3 Khâu đạo hàm (Derivative)



Hình 3-8 Sơ đồ khối khâu tích phân

Tốc độ thay đổi của sai số quá trình được tính toán bằng cách xác định độ dốc của sai số theo thời gian (tức là đạo hàm bậc một theo thời gian) và nhân tốc độ này với độ lợi tỉ lệ K_d. Biên độ của phân phối khâu vi phân (đôi khi được gọi là tốc độ) trên tất cả các hành vi điều khiển được giới hạn bởi độ lợi vi phân K_d.

- Hàm truyền:

$$K(s)=K_i.s$$

❖ Đặc tính thời gian:

$$Y(s)=K_s.G(s).E(s).s$$

❖ Sai số hệ thống:

$$E(s)=\frac{R(s)}{1+sK_dG(s)H(s)}$$

Khâu vi phân làm chậm tốc độ thay đổi của đầu ra bộ điều khiển và đặc tính này là đang chú ý nhất để đạt tới điểm đặt của bộ điều khiển. Từ đó, điều khiển vi phân được sử dụng để làm giảm biên độ vọt lố được tạo ra bởi thành phần tích phân và tăng cường độ ổn định của bộ điều khiển hỗn hợp. Tuy nhiên, phép vi phân của một tín hiệu sẽ khuếch đại nhiễu và do đó khâu này sẽ nhạy hơn đối với nhiễu trong sai số, và có thể khiến quá trình trở nên không ổn định nếu nhiễu và độ lợi vi phân đủ lớn. Do đó một xấp xỉ của bộ vi sai với băng thông giới hạn thường được sử dụng hơn. Chẳng hạn như mạch bù sớm pha.

Kết luận:

- Độ lợi tỉ lệ K_p: giá trị càng lớn thì đáp ứng càng nhanh do đó sai số càng lớn, bù khâu tỉ lệ càng lớn. Một giá trị độ lợi tỉ lệ quá lớn sẽ dẫn đến quá trình mất ổn định và dao động.
- Độ lợi tích phân K_i: giá trị càng lớn kéo theo sai số ổn định bị khử càng nhanh. Đổi lại là độ vọt lố càng lớn: bất kỳ sai số âm nào được tích phân trong suốt đáp ứng quá độ phải được triệt tiêu tích phân bằng sai số dương trước khi tiến tới trạng thái ổn định.
- Độ lợi vi phân K_d: giá trị càng lớn càng giảm độ vọt lố, nhưng lại làm chậm đáp ứng quá độ và có thể dẫn đến mất ổn định do khuếch đại nhiễu tín hiệu trong phép vi phân sai số.

CHƯƠNG 4. PHƯƠNG PHÁP ĐIỀU KHIỂN TỐC ĐỘ ĐỘNG CƠ

4.1 Điều khiển tốc độ động cơ theo phương pháp điều chế độ rộng xung PWM

Phương pháp điều khiển tốc độ động cơ DC bằng PWM (Pulse Width Modulation) là phương pháp điều chỉnh tốc độ động cơ DC bằng cách thay đổi độ rộng xung (pulse width) của tín hiệu điện áp cấp cho động cơ.

Tốc độ của động cơ DC được xác định bởi điện áp cấp cho động cơ và điện trở của dây quấn stato. Điện áp cấp cho động cơ càng cao thì tốc độ của động cơ càng lớn. Điện trở của dây quấn stato càng cao thì tốc độ của động cơ càng nhỏ.

Trong phương pháp điều khiển tốc độ động cơ DC bằng PWM, điện áp cấp cho động cơ được điều khiển bằng cách thay đổi độ rộng xung của tín hiệu PWM. Độ rộng xung là tỷ lệ thời gian mà tín hiệu PWM ở mức cao so với thời gian tổng của tín hiệu PWM.Ví dụ, nếu độ rộng xung của tín hiệu PWM là 50%, thì điện áp cấp cho động cơ sẽ bằng 50% điện áp định mức.

Để điều khiển tốc độ của động cơ DC bằng PWM, cần sử dụng một bộ điều khiển tốc độ PWM. Bộ điều khiển tốc độ PWM sẽ nhận tín hiệu điều khiển tốc độ từ người dùng và sử dụng tín hiệu này để điều chỉnh độ rộng xung của tín hiệu PWM.

Dưới đây là một ví dụ về cách điều khiển tốc độ động cơ DC bằng PWM:

- Nếu muốn tăng tốc độ của động cơ, cần tăng độ rộng xung của tín hiệu PWM.
- Nếu muốn giảm tốc độ của động cơ, cần giảm độ rộng xung của tín hiệu PWM.
- Nếu muốn dừng động cơ, cần đặt độ rộng xung của tín hiệu PWM bằng 0.

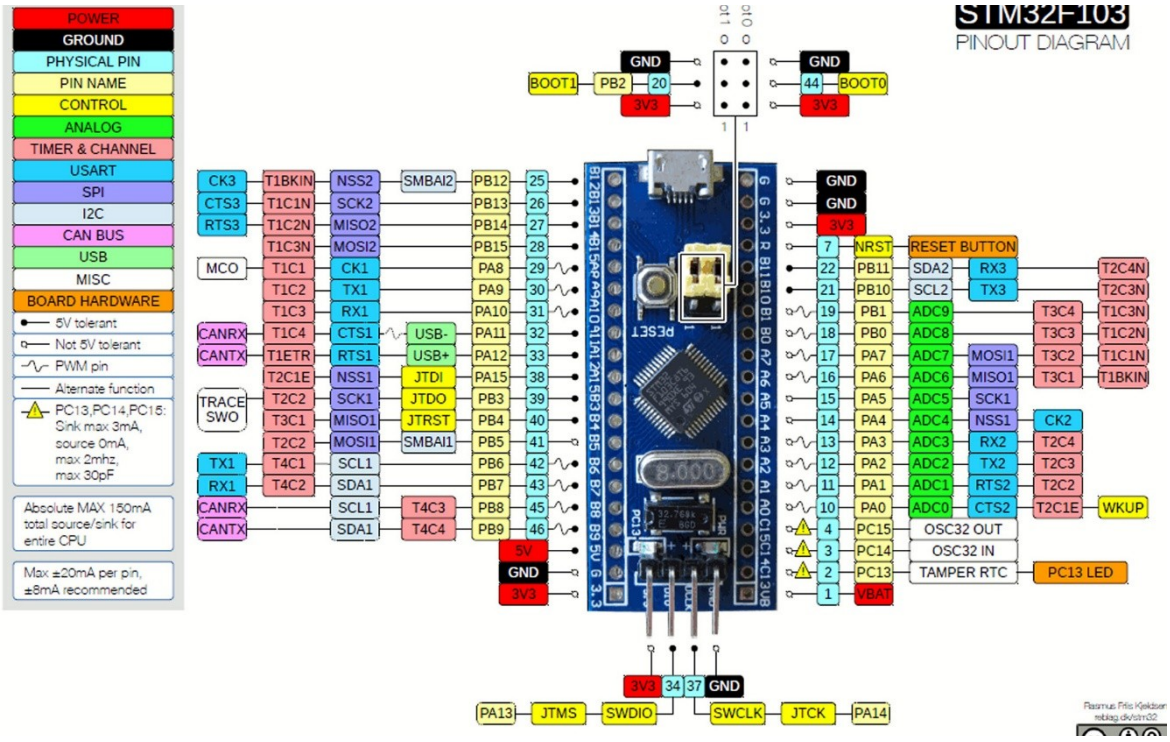
Phương pháp điều khiển tốc độ động cơ DC bằng PWM có nhiều ưu điểm, bao gồm: Độ chính xác cao, Chi phí thấp, Có thể điều khiển tốc độ động cơ một cách liên tục

Phương pháp này được sử dụng rộng rãi trong các ứng dụng công nghiệp và gia dụng.

Một số ứng dụng của phương pháp điều khiển tốc độ động cơ DC bằng PWM bao gồm: Điều khiển tốc độ quạt, Điều khiển tốc độ máy bơm, Điều khiển tốc độ máy CNC, Điều khiển tốc độ robot

4.2 Lập trình STM32 với PWM

4.2.1 Giới thiệu STM32F103C8T6



Hình 4-9 Sơ đồ module STM32F103C6T8

Dòng sản phẩm STM32F103x4 và STM32F103x6 thuộc dòng hiệu suất cao tích hợp nhân ARM® Cortex™-M3 32-bit RISC mạnh mẽ hoạt động ở tần số 72 MHz, bộ nhớ nhúng tốc độ cao (bộ nhớ Flash lên đến 32 Kbyte và SRAM lên đến 6 Kbyte) và nhiều I/O và ngoại vi nâng cao được kết nối với hai bus APB. Tất cả các thiết bị đều cung cấp hai ADC 12-bit, ba bộ đếm 16-bit đa năng cộng với một bộ đếm PWM, cũng như các giao diện truyền thông tiêu chuẩn và nâng cao: tối đa hai I2C và SPI, ba USART, một USB và một CAN.

Dòng sản phẩm STM32F103xx hoạt động với nguồn cung cấp điện áp từ 2.0 đến 3.6 V. Nó có sẵn ở cả dải nhiệt độ từ –40 đến +85 °C và dải nhiệt độ mở rộng từ –40 đến +105 °C. Một bộ chế độ tiết kiệm điện toàn diện cho phép thiết kế các ứng dụng công suất thấp.

Dòng sản phẩm STM32F103xx bao gồm các thiết bị trong bốn loại gói khác nhau: từ 36 chân đến 64 chân. Tùy thuộc vào thiết bị được chọn, các bộ ngoại vi khác nhau sẽ được bao gồm, mô tả dưới đây cung cấp tổng quan về toàn bộ phạm vi ngoại vi được đề xuất trong dòng sản phẩm này.

Nhờ những tính năng này mà dòng sản phẩm vi điều khiển STM32F103xx low-density performance line phù hợp với nhiều ứng dụng khác nhau như điều khiển động cơ, điều khiển ứng dụng, thiết bị y tế và cầm tay, thiết bị ngoại vi PC và trò chơi, nền tảng GPS, ứng dụng công nghiệp, PLC, biến tần, máy in, máy quét, hệ thống báo động, video intercom và HVAC.

ARM 32-bit Cortex™-M3 CPU Core

- 72 MHz maximum frequency,1.25 DMIPS/MHz (Dhrystone 2.1) performance at 0 wait state memory access
- Single-cycle multiplication and hardware division

Memories

- 16 or 32 Kbytes of Flash memory
- 6 or 10 Kbytes of SRAM

Clock, reset and supply management

- 2.0 to 3.6 V application supply and I/Os
- POR, PDR, and programmable voltage detector (PVD)
- 4-to-16 MHz crystal oscillator
- Internal 8 MHz factory-trimmed RC
- Internal 40 kHz RC
- PLL for CPU clock
- 32 kHz oscillator for RTC with calibration

Low power: Sleep, Stop and Standby modes

VBAT supply for RTC and backup registers

2 x 12-bit, 1 µs A/D converters (up to 16 channels)

- Conversion range: 0 to 3.6 V
- Dual-sample and hold capability
- Temperature sensor

DMA

- 7-channel DMA controller
- Peripherals supported: timers, ADC, SPIs, I2Cs and USARTs

Up to 51 fast I/O ports

- 26/37/51 I/Os, all mappable on 16 external interrupt vectors and almost all 5 V-tolerant

Debug mode

- Serial wire debug (SWD) & JTAG interfaces

6 timers

- Two 16-bit timers, each with up to 4 IC/OC/PWM or pulse counter and quadrature (incremental) encoder input
- 16-bit, motor control PWM timer with dead-time generation and emergency stop
- 2 watchdog timers (Independent and Window)
- SysTick timer 24-bit downcounter

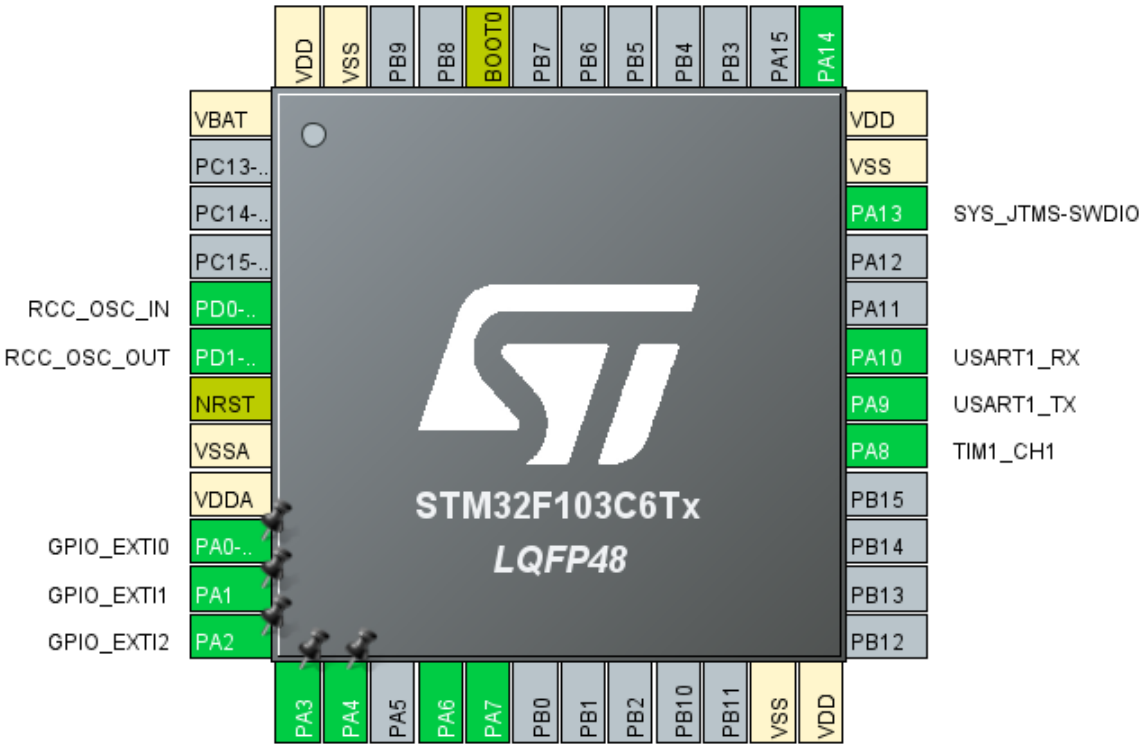
6 communication interfaces

- 1 x I2C interface (SMBus/PMBus)
- 2 × USARTs (ISO 7816 interface, LIN, IrDA capability, modem control)
- 1 × SPI (18 Mbit/s)
- CAN interface (2.0B Active)
- USB 2.0 full-speed interface

CRC calculation unit, 96-bit unique ID

Packages are ECOPACK®

4.2.2 Cấu hình cho vi điều khiển



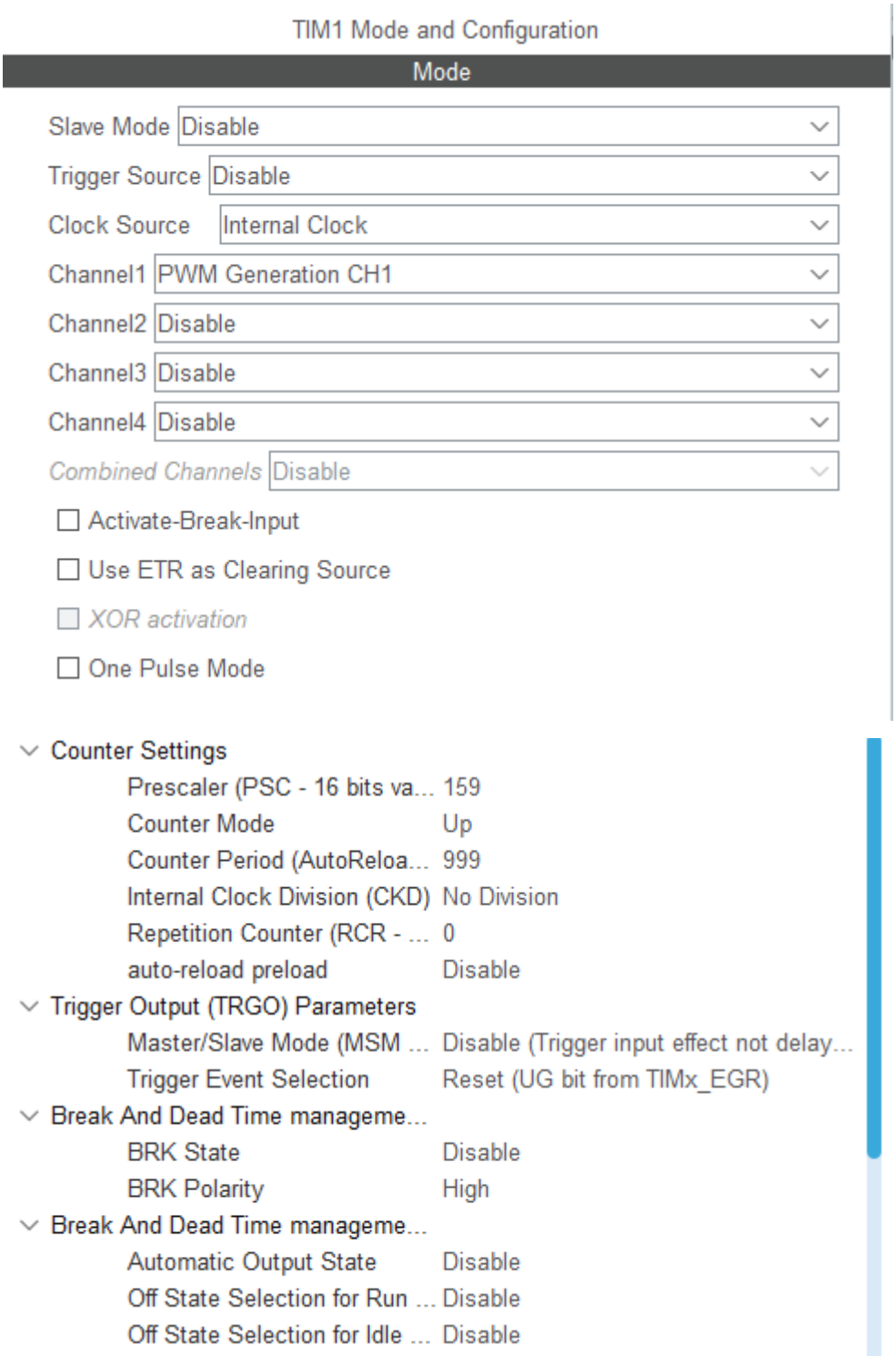
Hình 4-10 Cấu hình chân STM32F103C6T8

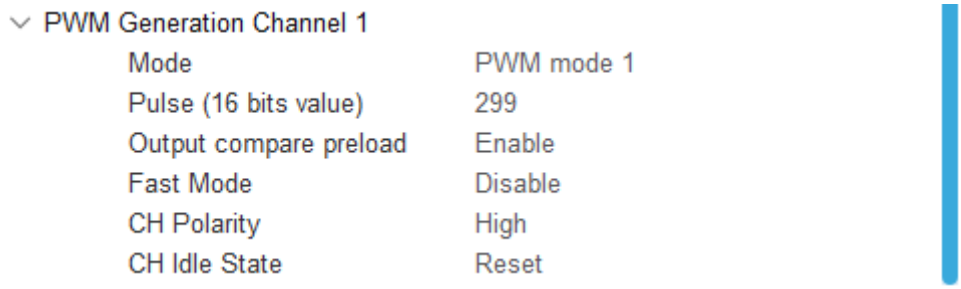
❖ Ngắt ngoài:

- Pin PA0 – GPIO_EXTI0: Nút nhấn START, có chức năng cho phép module L298 hoạt động (kích mức cao cho chân En L298).
- Pin PA1 – GPIO_EXTI1: Nút nhấn UP, có chức năng tăng giá trị vận tốc.
- Pin PA2 – GPIO_EXTI2: Nút nhấn DOWN, có chức năng giảm giá trị vận tốc.
- Pin PA3 – GPIO_EXTI3: Nút nhấn EXIT, có chức năng thoát các lựa chọn, dừng hệ thống.

❖ PWM:

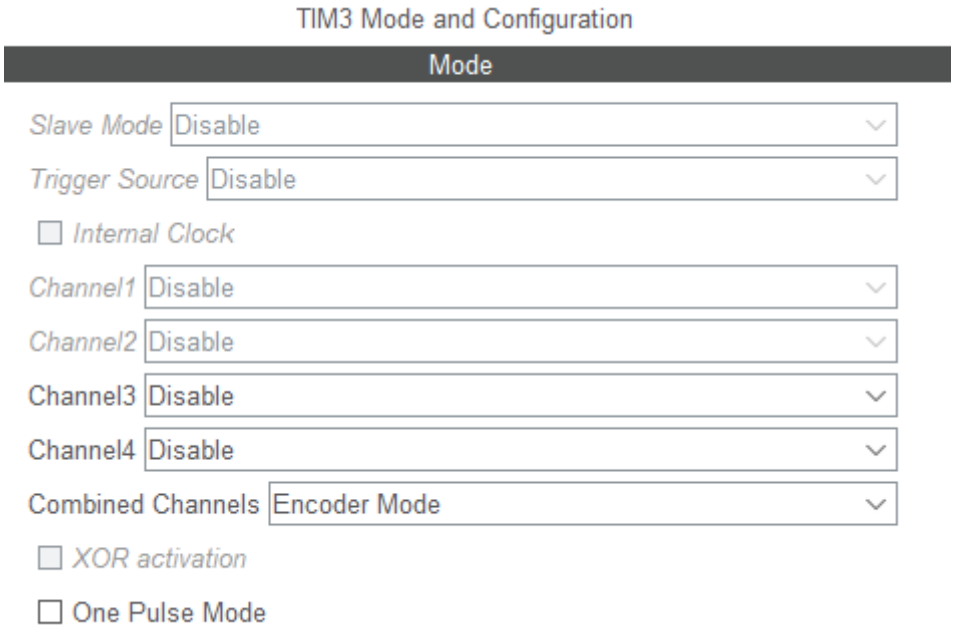
Pin PA8 – TIMER_CHANNEL_1:


TIM1 Mode and Configuration
Mode
Slave Mode Disable
Trigger Source Disable
Clock Source Internal Clock
Channel1 PWM Generation CH1
Channel2 Disable
Channel3 Disable
Channel4 Disable
Combined Channels Disable
Activate-Break-Input
Use ETR as Clearing Source
XOR activation
One Pulse Mode
Counter Settings
Prescaler (PSC - 16 bits va... 159
Counter Mode Up
Counter Period (AutoReloa... 999
Internal Clock Division (CKD) No Division
Repetition Counter (RCR - ... 0
auto-reload preload Disable
Trigger Output (TRGO) Parameters
Master/Slave Mode (MSM ... Disable (Trigger input effect not delay...
Trigger Event Selection Reset (UG bit from TIMx_EGR)
Break And Dead Time manageme...
BRK State Disable
BRK Polarity High
Break And Dead Time manageme...
Automatic Output State Disable
Off State Selection for Run ... Disable
Off State Selection for Idle ... Disable



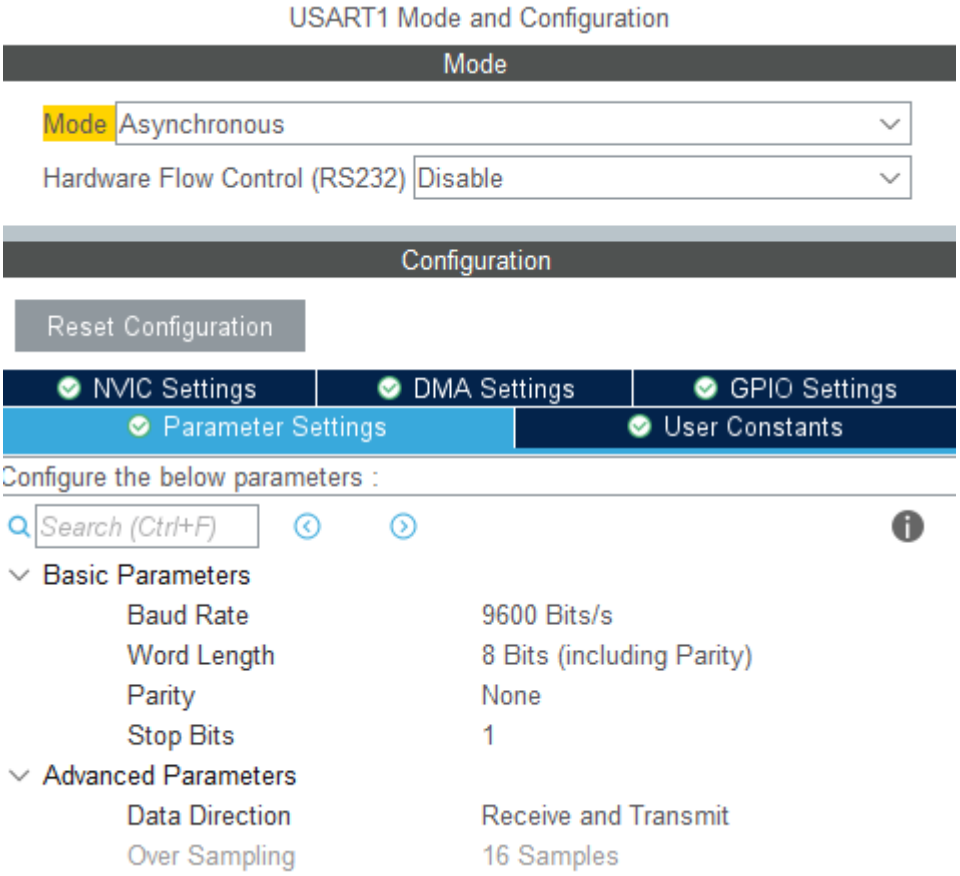
❖ Đọc Encoder:

- Pin PA6 – TIMER3_CHANNEL_1: nối với Cực A Encoder
- Pin PA7 – TIMER3_CHANNEL_2: nối với Cực B Encoder



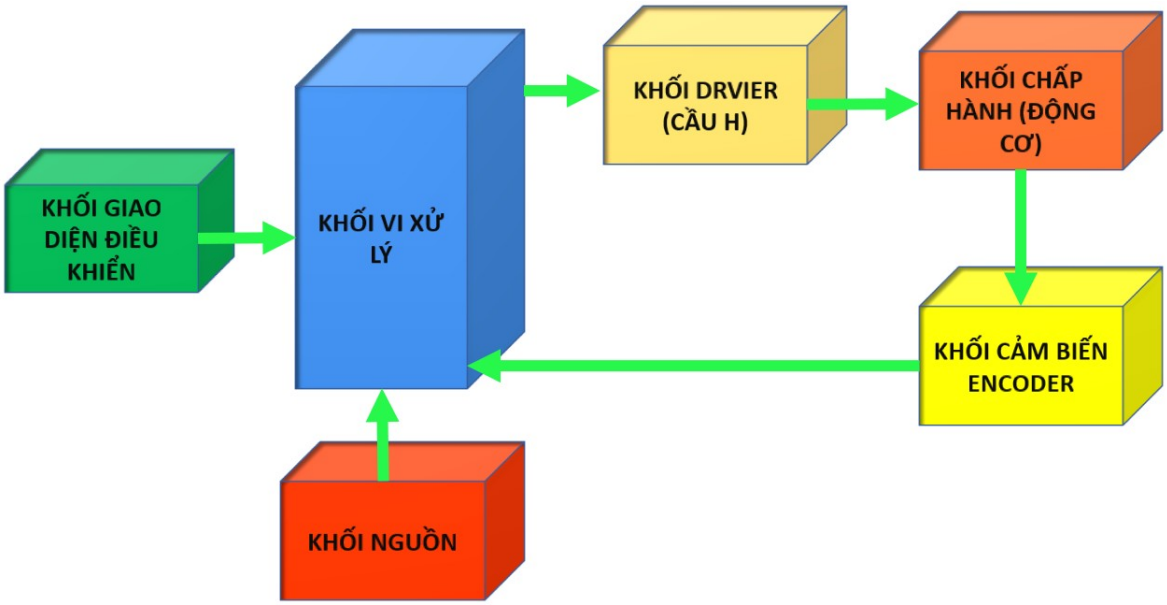
❖ Truyền thông nối tiếp dữ liệu:

- Pin PA9 – USART1_TX: nối với Pin D4 (RX) của module ESP8266
- Pin PA10 – USART1_RX: nối với Pin D5 (TX) của module ESP8266


USART1 Mode and Configuration
Mode
Mode
Asynchronous
Hardware Flow Control (RS232)
Disable
Configuration
Reset Configuration
NVIC Settings
DMA Settings
GPIO Settings
Parameter Settings
User Constants
Configure the below parameters :
Search (Ctrl+F)
Basic Parameters
Baud Rate
9600 Bits/s
Word Length
8 Bits (including Parity)
Parity
None
Stop Bits
1
Advanced Parameters
Data Direction
Receive and Transmit
Over Sampling
16 Samples

CHƯƠNG 5. THIẾT KẾ MÔ HÌNH

5.1 Sơ đồ khối của hệ thống

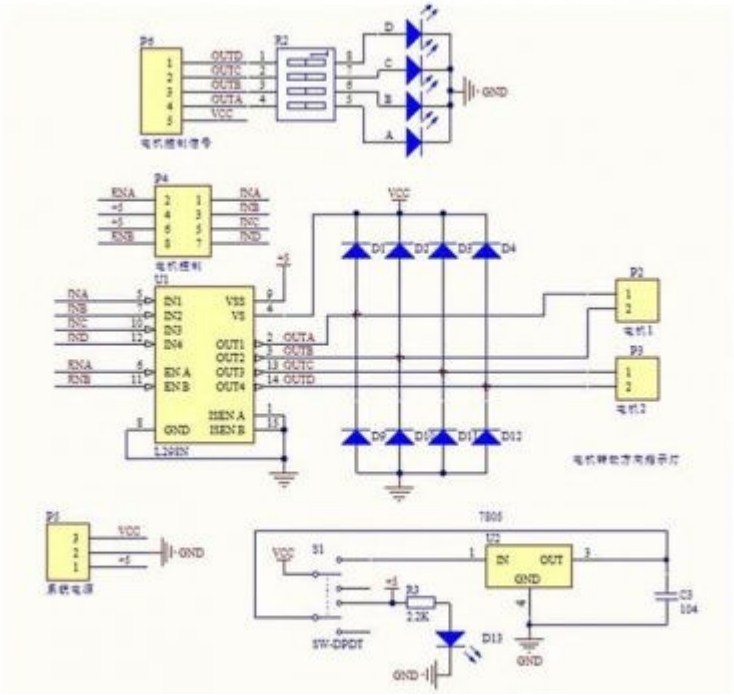


Hình 6-11 Sơ đồ khối của hệ thống

Khối giao diện điều khiển: có chức năng giao tiếp với người dùng, hiển thị tốc độ đo được của động cơ bằng Encoder, người dùng có thể cài đặt tốc độ mong muốn.

Khối vi xử lý: là bộ não của hệ thống, có chức năng điều khiển, xử lý mọi sự kiện dựa vào thuật toán đã được thiết lập. Song, nó còn có chức năng truyền, nhận dữ liệu cới cơ cấu chấp hành và giám sát hệ thống.

Khối Driver: IC L298 có thể được sử dụng để điều khiển động cơ DC theo nhiều cách khác nhau. Một cách phổ biến là sử dụng phương pháp điều khiển tốc độ động cơ DC bằng PWM. Trong phương pháp này, IC L298 sẽ sử dụng tín hiệu PWM để điều khiển tốc độ của động cơ DC.

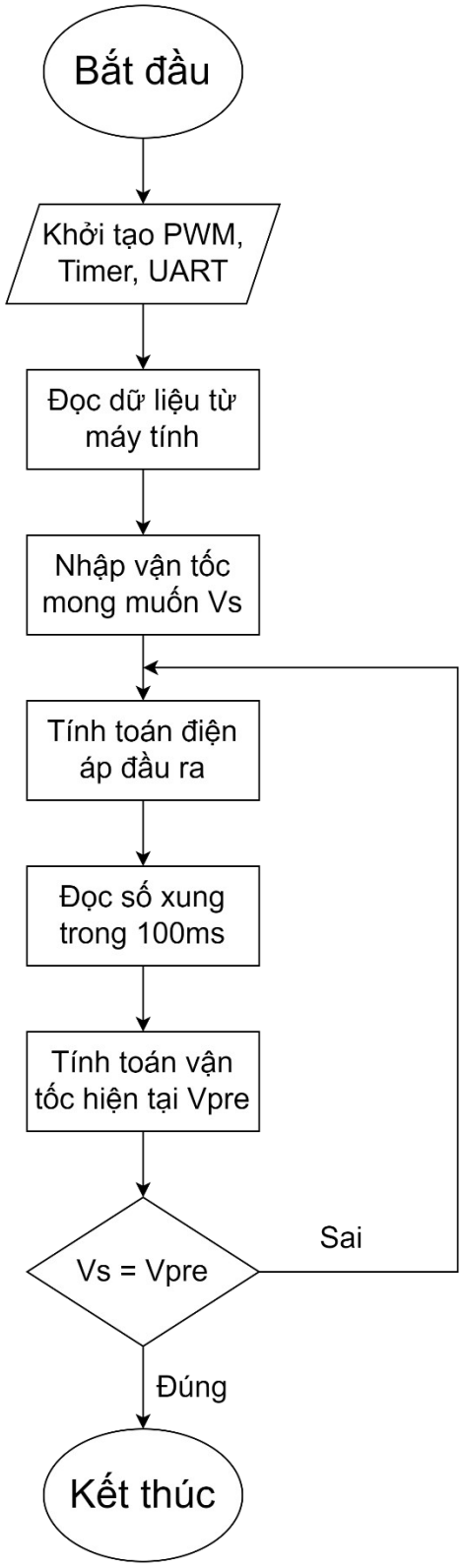


Hình 6-12 Sơ đồ nguyên lý module L298

Khối cơ cấu chấp hành là động cơ DC sẽ hoạt động dựa theo điện áp được điều chế bằng khối Driver.

Khối cảm biến: có chức năng phản hồi tốc độ, cụ thể là Encoder sẽ đếm số xung và trả về khối vi xử lý, từ đó tính toán tốc độ của động cơ nhằm duy trì được tốc độ mong muốn mà người dùng đã cài đặt.

5.2 Lưu đồ giải thuật



Hình 6-13 Lưu đồ giải thuật

5.3 Chương trình lập trình

5.3.1 Thư viện LCD

```
#ifndef __LCD_16X2_H__
#define __LCD_16X2_H__
#include "stm32f1xx_hal.h"
#include "string.h"
/* * * *
```

```
#define LINES                                   2
// DDRAM address of first char of line 1
#define START_LINE_1                    0x00
// DDRAM address of first char of line 2
#define START_LINE_2                    0x40
#define SET_DDRAM_ADDRESS               0x80
#define LCD16xN             // For 16xN LCDs
/******** Command register *********************/
#define CLEAR_DISPLAY                   0x01
#define RETURN_HOME                 0x02
#define ENTRY_MODE_SET          0x04
#define OPT_S               0x01                // Shift entire display to right
#define OPT_INC                     0x02                // Cursor increment
#define DISPLAY_ON_OFF_CONTROL  0x08
#define OPT_D               0x04                // Turn on display
#define OPT_C                                   0x02                // Turn on
cursor
#define OPT_B                                   0x01                // Turn on cursor
blink
#define CURSOR_DISPLAY_SHIFT    0x10    // Move and shift cursor
#define OPT_SC                                  0x08
#define OPT_RL                                  0x04
#define FUNCTION_SET                0x20
#define OPT_DL                                  0x10                //      Set
interface data length
#define OPT_N                                   0x08                //      Set
number of display lines
#define OPT_F                                   0x04                //      Set
alternate font
```

```
#define SET_DDRAM_ADDR                  0x80            // Set DDRAM
address
/*************** LCD defines **************/
#define LCD_NIB                 4
#define LCD_BYTE                    8
#define LCD_DATA_REG        GPIO_PIN_SET
#define LCD_COMMAND_REG     GPIO_PIN_RESET
/****************** LCD typedefs **********************/
#define Lcd_PortType  GPIO_TypeDef*
#define Lcd_PinType   uint16_t
/*********** Public functions **********************/
void    Lcd_Init(void);
void    Lcd_clear_xy(uint8_t x,uint8_t y);
void    Lcd_write_custom_char( uint8_t x, uint8_t y, uint8_t location);
void    Lcd_gotoxy( uint8_t x, uint8_t y);
void    Lcd_create_custom_char(uint8_t location, uint8_t* data_bytes);
void    Lcd_clear_display(void);
void    Lcd_write_string(char * string);
void    Lcd_write_int(int number);
#endif /* __LCD_16X2_H__ */
```

5.3.2 Thư viện PID

```
#ifndef __PID_H
#define __PID_H
#ifdef __cplusplus
extern "C" {
#endif
/* Includes ----------------------*/
#include "main.h"
```

```
typedef struct
{
   float A0;         /**< The derived gain, A0 = Kp + Ki + Kd . */
   float A1;         /**< The derived gain, A1 = -Kp - 2Kd. */
   float A2;         /**< The derived gain, A2 = Kd . */
   float state[3];    /**< The state array of length 3. */
   float Kp;         /**< The proportional gain. */
   float Ki;         /**< The integral gain. */
   float Kd;         /**< The derivative gain. */
     float SP;
} arm_pid_instance_f32;
  /**
   * @brief  Initialization function for the floating-point PID Control.
   * @param[in,out] S               points to an instance of the PID structure.
   * @param[in]     resetStateFlag  flag to reset the state. 0 = no change in state 1 =
reset the state.
   */
void pid_init(arm_pid_instance_f32 * S,float Kp,float Ki, float Kd);
float pid_cal(arm_pid_instance_f32 * S, float in);
void pid_setPoint(arm_pid_instance_f32 * S,float sp);
  /**
   * @brief  Reset function for the floating-point PID Control.
   * @param[in,out] S  is an instance of the floating-point PID Control structure
   */
void pid_reset(arm_pid_instance_f32 * S);
#ifdef __cplusplus
}
#endif
#endif /* __MAIN_H */
```

5.3.3 Chương trình chính

```
/* USER CODE BEGIN Header */
/**
  ******************************************************************************
  * @file           : main.c
  * @brief          : Main program body
  ******************************************************************************
  * @attention
  *
  * Copyright (c) 2023 STMicroelectronics.
  * All rights reserved.
  *
  * This software is licensed under terms that can be found in the LICENSE file
  * in the root directory of this software component.
  * If no LICENSE file comes with this software, it is provided AS-IS.
  *
  ******************************************************************************
  */
/* USER CODE END Header */
/* Includes ------------------------------------------------------------------*/
#include "main.h"

/* Private includes ----------------------------------------------------------*/
/* USER CODE BEGIN Includes */
#include "stdio.h"
#include "lcd_16x2.h"
```

```
#include "PID.h"
#include "string.h"
/* USER CODE END Includes */

/* Private typedef -----------------------------------------------------------*/
/* USER CODE BEGIN PTD */

/* USER CODE END PTD */

/* Private define ------------------------------------------------------------*/
/* USER CODE BEGIN PD */
/* USER CODE END PD */

/* Private macro -------------------------------------------------------------*/
/* USER CODE BEGIN PM */

/* USER CODE END PM */

/* Private variables ---------------------------------------------------------*/
TIM_HandleTypeDef htim1;
TIM_HandleTypeDef htim3;

UART_HandleTypeDef huart1;

/* USER CODE BEGIN PV */
uint16_t var;
uint16_t encoder_cnt_pre =0;
uint16_t encoder_cnt =0;
uint8_t msg[35]={'\0'};
```

```
uint8_t speed=0;
/* USER CODE END PV */

/* Private function prototypes -----------------------------------------------*/
void SystemClock_Config(void);
static void MX_GPIO_Init(void);
static void MX_TIM1_Init(void);
static void MX_TIM3_Init(void);
static void MX_USART1_UART_Init(void);
/* USER CODE BEGIN PFP */

/* USER CODE END PFP */

/* Private user code ---------------------------------------------------------*/
/* USER CODE BEGIN 0 */

/* USER CODE END 0 */

/**
  * @brief  The application entry point.
  * @retval int
  */
int main(void)
{
  /* USER CODE BEGIN 1 */

  /* USER CODE END 1 */

  /* MCU Configuration--------------------------------------------------------*/
```

```
/* Reset of all peripherals, Initializes the Flash interface and the Systick. */
HAL_Init();

/* USER CODE BEGIN Init */
  Lcd_Init();
/* USER CODE END Init */

/* Configure the system clock */
SystemClock_Config();

/* USER CODE BEGIN SysInit */

/* USER CODE END SysInit */

/* Initialize all configured peripherals */
MX_GPIO_Init();
MX_TIM1_Init();
MX_TIM3_Init();
MX_USART1_UART_Init();
/* USER CODE BEGIN 2 */
HAL_TIM_PWM_Start(&htim1, TIM_CHANNEL_1);
HAL_TIM_Encoder_Start(&htim3, TIM_CHANNEL_1|TIM_CHANNEL_2);
/* USER CODE END 2 */
  Lcd_write_string("Do Minh Chinh");

/* Infinite loop */
/* USER CODE BEGIN WHILE */
while (1)
```

```
  {
    /* USER CODE END WHILE */

    /* USER CODE BEGIN 3 */
       HAL_Delay(1000);
            encoder_cnt=__HAL_TIM_GET_COUNTER(&htim3);
            speed=(encoder_cnt-encoder_cnt_pre)/100;
            encoder_cnt_pre =encoder_cnt;
            //truyen tin
            sprintf(msg,"speed = %d\r\n", speed);
            HAL_UART_Transmit(&huart1, msg, sizeof(msg), 1);
  }
  /* USER CODE END 3 */
}

/**
  * @brief System Clock Configuration
  * @retval None
  */
void SystemClock_Config(void)
{
  RCC_OscInitTypeDef RCC_OscInitStruct = {0};
  RCC_ClkInitTypeDef RCC_ClkInitStruct = {0};

  /** Initializes the RCC Oscillators according to the specified parameters
  * in the RCC_OscInitTypeDef structure.
  */
  RCC_OscInitStruct.OscillatorType = RCC_OSCILLATORTYPE_HSI;
  RCC_OscInitStruct.HSIState = RCC_HSI_ON;
```

```
                                RCC_OscInitStruct.HSICalibrationValue =
RCC_HSICALIBRATION_DEFAULT;
  RCC_OscInitStruct.PLL.PLLState = RCC_PLL_NONE;
  if (HAL_RCC_OscConfig(&RCC_OscInitStruct) != HAL_OK)
  {
    Error_Handler();
  }

  /** Initializes the CPU, AHB and APB buses clocks
  */
            RCC_ClkInitStruct.ClockType      =      RCC_CLOCKTYPE_HCLK|
RCC_CLOCKTYPE_SYSCLK
                        |RCC_CLOCKTYPE_PCLK1|RCC_CLOCKTYPE_PCLK2;
  RCC_ClkInitStruct.SYSCLKSource = RCC_SYSCLKSOURCE_HSI;
  RCC_ClkInitStruct.AHBCLKDivider = RCC_SYSCLK_DIV1;
  RCC_ClkInitStruct.APB1CLKDivider = RCC_HCLK_DIV1;
  RCC_ClkInitStruct.APB2CLKDivider = RCC_HCLK_DIV1;

    if (HAL_RCC_ClockConfig(&RCC_ClkInitStruct, FLASH_LATENCY_0) !=
HAL_OK)
  {
    Error_Handler();
  }
}

/**
  * @brief TIM1 Initialization Function
  * @param None
  * @retval None
```

```
 */
static void MX_TIM1_Init(void)
{

 /* USER CODE BEGIN TIM1_Init 0 */

 /* USER CODE END TIM1_Init 0 */

 TIM_ClockConfigTypeDef sClockSourceConfig = {0};
 TIM_MasterConfigTypeDef sMasterConfig = {0};
 TIM_OC_InitTypeDef sConfigOC = {0};
 TIM_BreakDeadTimeConfigTypeDef sBreakDeadTimeConfig = {0};

 /* USER CODE BEGIN TIM1_Init 1 */

 /* USER CODE END TIM1_Init 1 */
 htim1.Instance = TIM1;
 htim1.Init.Prescaler = 159;
 htim1.Init.CounterMode = TIM_COUNTERMODE_UP;
 htim1.Init.Period = 999;
 htim1.Init.ClockDivision = TIM_CLOCKDIVISION_DIV1;
 htim1.Init.RepetitionCounter = 0;
 htim1.Init.AutoReloadPreload = TIM_AUTORELOAD_PRELOAD_DISABLE;
 if (HAL_TIM_Base_Init(&htim1) != HAL_OK)
 {
  Error_Handler();
 }
 sClockSourceConfig.ClockSource = TIM_CLOCKSOURCE_INTERNAL;
 if (HAL_TIM_ConfigClockSource(&htim1, &sClockSourceConfig) != HAL_OK)
```

```
  {
    Error_Handler();
  }
  if (HAL_TIM_PWM_Init(&htim1) != HAL_OK)
  {
    Error_Handler();
  }
  sMasterConfig.MasterOutputTrigger = TIM_TRGO_RESET;
  sMasterConfig.MasterSlaveMode = TIM_MASTERSLAVEMODE_DISABLE;
    if (HAL_TIMEx_MasterConfigSynchronization(&htim1, &sMasterConfig) !=
HAL_OK)
  {
    Error_Handler();
  }
  sConfigOC.OCMode = TIM_OCMODE_PWM1;
  sConfigOC.Pulse = 299;
  sConfigOC.OCPolarity = TIM_OCPOLARITY_HIGH;
  sConfigOC.OCNPolarity = TIM_OCNPOLARITY_HIGH;
  sConfigOC.OCFastMode = TIM_OCFAST_DISABLE;
  sConfigOC.OCIdleState = TIM_OCIDLESTATE_RESET;
  sConfigOC.OCNIdleState = TIM_OCNIDLESTATE_RESET;
              if (HAL_TIM_PWM_ConfigChannel(&htim1, &sConfigOC,
TIM_CHANNEL_1) != HAL_OK)
  {
    Error_Handler();
  }
  sBreakDeadTimeConfig.OffStateRunMode = TIM_OSSR_DISABLE;
  sBreakDeadTimeConfig.OffStateIDLEMode = TIM_OSSI_DISABLE;
  sBreakDeadTimeConfig.LockLevel = TIM_LOCKLEVEL_OFF;
```

```
  sBreakDeadTimeConfig.DeadTime = 0;
  sBreakDeadTimeConfig.BreakState = TIM_BREAK_DISABLE;
  sBreakDeadTimeConfig.BreakPolarity = TIM_BREAKPOLARITY_HIGH;
                                    sBreakDeadTimeConfig.AutomaticOutput                =
TIM_AUTOMATICOUTPUT_DISABLE;
   if (HAL_TIMEx_ConfigBreakDeadTime(&htim1, &sBreakDeadTimeConfig) !=
HAL_OK)
  {
    Error_Handler();
  }
  /* USER CODE BEGIN TIM1_Init 2 */

  /* USER CODE END TIM1_Init 2 */
  HAL_TIM_MspPostInit(&htim1);

}

/**
  * @brief TIM3 Initialization Function
  * @param None
  * @retval None
  */
static void MX_TIM3_Init(void)
{

  /* USER CODE BEGIN TIM3_Init 0 */

  /* USER CODE END TIM3_Init 0 */
```

```
TIM_Encoder_InitTypeDef sConfig = {0};
TIM_MasterConfigTypeDef sMasterConfig = {0};

/* USER CODE BEGIN TIM3_Init 1 */

/* USER CODE END TIM3_Init 1 */
htim3.Instance = TIM3;
htim3.Init.Prescaler = 0;
htim3.Init.CounterMode = TIM_COUNTERMODE_UP;
htim3.Init.Period = 65535;
htim3.Init.ClockDivision = TIM_CLOCKDIVISION_DIV1;
htim3.Init.AutoReloadPreload = TIM_AUTORELOAD_PRELOAD_DISABLE;
sConfig.EncoderMode = TIM_ENCODERMODE_TI1;
sConfig.IC1Polarity = TIM_ICPOLARITY_RISING;
sConfig.IC1Selection = TIM_ICSELECTION_DIRECTTI;
sConfig.IC1Prescaler = TIM_ICPSC_DIV1;
sConfig.IC1Filter = 0;
sConfig.IC2Polarity = TIM_ICPOLARITY_RISING;
sConfig.IC2Selection = TIM_ICSELECTION_DIRECTTI;
sConfig.IC2Prescaler = TIM_ICPSC_DIV1;
sConfig.IC2Filter = 0;
if (HAL_TIM_Encoder_Init(&htim3, &sConfig) != HAL_OK)
{
  Error_Handler();
}
sMasterConfig.MasterOutputTrigger = TIM_TRGO_RESET;
sMasterConfig.MasterSlaveMode = TIM_MASTERSLAVEMODE_DISABLE;
  if (HAL_TIMEx_MasterConfigSynchronization(&htim3, &sMasterConfig) !=
HAL_OK)
```

```
  {
    Error_Handler();
  }
  /* USER CODE BEGIN TIM3_Init 2 */

  /* USER CODE END TIM3_Init 2 */

}

/**
  * @brief USART1 Initialization Function
  * @param None
  * @retval None
  */
static void MX_USART1_UART_Init(void)
{

  /* USER CODE BEGIN USART1_Init 0 */

  /* USER CODE END USART1_Init 0 */

  /* USER CODE BEGIN USART1_Init 1 */

  /* USER CODE END USART1_Init 1 */
  huart1.Instance = USART1;
  huart1.Init.BaudRate = 9600;
  huart1.Init.WordLength = UART_WORDLENGTH_8B;
  huart1.Init.StopBits = UART_STOPBITS_1;
  huart1.Init.Parity = UART_PARITY_NONE;
```

```
  huart1.Init.Mode = UART_MODE_TX_RX;
  huart1.Init.HwFlowCtl = UART_HWCONTROL_NONE;
  huart1.Init.OverSampling = UART_OVERSAMPLING_16;
  if (HAL_UART_Init(&huart1) != HAL_OK)
  {
    Error_Handler();
  }
  /* USER CODE BEGIN USART1_Init 2 */

  /* USER CODE END USART1_Init 2 */

}

/**
  * @brief GPIO Initialization Function
  * @param None
  * @retval None
  */
static void MX_GPIO_Init(void)
{
  GPIO_InitTypeDef GPIO_InitStruct = {0};

  /* GPIO Ports Clock Enable */
  __HAL_RCC_GPIOD_CLK_ENABLE();
  __HAL_RCC_GPIOA_CLK_ENABLE();

  /*Configure GPIO pin Output Level */
  HAL_GPIO_WritePin(GPIOA, GPIO_PIN_3|GPIO_PIN_4, GPIO_PIN_RESET);
```

```
  /*Configure GPIO pins : PA0 PA1 PA2 */
  GPIO_InitStruct.Pin = GPIO_PIN_0|GPIO_PIN_1|GPIO_PIN_2;
  GPIO_InitStruct.Mode = GPIO_MODE_IT_RISING;
  GPIO_InitStruct.Pull = GPIO_PULLUP;
  HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);

  /*Configure GPIO pins : PA3 PA4 */
  GPIO_InitStruct.Pin = GPIO_PIN_3|GPIO_PIN_4;
  GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
  GPIO_InitStruct.Pull = GPIO_NOPULL;
  GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
  HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);

  /* EXTI interrupt init*/
  HAL_NVIC_SetPriority(EXTI0_IRQn, 2, 0);
  HAL_NVIC_EnableIRQ(EXTI0_IRQn);

  HAL_NVIC_SetPriority(EXTI1_IRQn, 2, 0);
  HAL_NVIC_EnableIRQ(EXTI1_IRQn);

  HAL_NVIC_SetPriority(EXTI2_IRQn, 2, 0);
  HAL_NVIC_EnableIRQ(EXTI2_IRQn);

}

/* USER CODE BEGIN 4 */
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
{
    if(GPIO_Pin==GPIO_PIN_0)
```

```
      {
         while(HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_2)==0){}
           HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_3);
          var=100;
      }
      if(GPIO_Pin==GPIO_PIN_1)
          {
            while(HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_0)==0){}
            if(var<900)
            var=var+400;
          }
       if(GPIO_Pin==GPIO_PIN_2)
          {
            while(HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_1)==0){}
            if(var>100)
            var=var-400;
          }
       __HAL_TIM_SET_COMPARE(&htim1,TIM_CHANNEL_1,var);
}
void pid_init(arm_pid_instance_f32 * S,float Kp,float Ki, float Kd)
{
  S->Kp = Kp;
  S->Ki = Kd;
  S->Kd = Kd;
 /* Derived coefficient A0 */
 S->A0 = S->Kp + S->Ki + S->Kd;

 /* Derived coefficient A1 */
 S->A1 = (-S->Kp) - ((float) 2.0 * S->Kd);
```

```
  /* Derived coefficient A2 */
  S->A2 = S->Kd;

  /* Check whether state needs reset or not */

}

void pid_setPoint(arm_pid_instance_f32 * S,float sp)
{
   S->SP = sp;
}
  /**
   * @brief  Reset function for the floating-point PID Control.
   * @param[in,out] S  is an instance of the floating-point PID Control structure
   */
void pid_reset(arm_pid_instance_f32 * S)
{

  /* Clear the state buffer.  The size will be always 3 samples */
  memset(S->state, 0, 3U * sizeof(float));
}

float pid_cal(arm_pid_instance_f32 * S, float in)
{
   float out;
   float err = S->SP - in;
   /* y[n] = y[n-1] + A0 * x[n] + A1 * x[n-1] + A2 * x[n-2]  */
   out = S->state[2] + S->A0 * err + S->A1 * S->state[0] + S->A2 * S->state[1];
```

```
  /* Update state */
  S->state[1] = S->state[0];
  S->state[0] = err;
  S->state[2] = out;

  /* return to application */
  return (out);
}
/* USER CODE END 4 */

/**
  * @brief  This function is executed in case of error occurrence.
  * @retval None
  */
void Error_Handler(void)
{
  /* USER CODE BEGIN Error_Handler_Debug */
  /* User can add his own implementation to report the HAL error return state */
  __disable_irq();
  while (1)
  {
  }
  /* USER CODE END Error_Handler_Debug */
}

#ifdef  USE_FULL_ASSERT
/**
  * @brief  Reports the name of the source file and the source line number
```

```
  *         where the assert_param error has occurred.
  * @param  file: pointer to the source file name
  * @param  line: assert_param error line source number
  * @retval None
  */
void assert_failed(uint8_t *file, uint32_t line)
{
  /* USER CODE BEGIN 6 */
  /* User can add his own implementation to report the file name and line number,
     ex: printf("Wrong parameters value: file %s on line %d\r\n", file, line) */
  /* USER CODE END 6 */
}
#endif /* USE_FULL_ASSERT */
```

TÀI LIỆU THAM KHẢO

[1] Nguyễn Doãn Phước; Lý thuyết điều khiển tuyến tính; NXB khoa học và kỹ thuật, Hà Nội, 2002.

[2] Nguyễn Thương Ngô; Lý thuyết tự động thông thườngvà hiện đại - Quyển 1 hệ tuyến tính; NXB Khoa học và kỹ thuật, Hà Nội, 2005.

[3] Nguyễn Văn Hoà; Cơ sở lý thuyết điều khiển tự động; NXB Khoa học và kỹ thuật, Hà Nội, 1998.

[4] Phạm Công Ngô; Lý thuyết điều khiển tự động; NXB Khoa học và kỹ thuật, Hà Nội, 1996.